

ASCURE Pilot Data Reset Plan

Remove all old pilot operational data and templates to start a clean pilot after Governance G3, while preserving tenant, org structure, MAINHEADs, teams, users, assignments, capabilities, and asset types.

Assessment and reset script only — do not execute.

Change ID: pilot-data-reset-20260604 | Target DB: ascure (container ascure-postgres, PostgreSQL 16) | 18 tables deleted, 24 kept. Verified: no KEEP table has a foreign key into any DELETE table, so the wipe cannot orphan or block kept data.

DESTRUCTIVE CHANGE — Take the Section 3 backup first. Run the Section 2 deletes inside the open transaction, verify with Section 5, and COMMIT only if correct; otherwise ROLLBACK. No separate checklist table exists: checklist templates ARE InspectionTemplate rows and are removed by the template deletes.

Keep / Delete map (physical tables)

KEEP (24): Tenant, Department, Organization, OrganizationCapability, OrganizationCapabilityAssignment, OrganizationMembership, Branch, BranchCapability, OperationalRegion, Mainhead, UserMainheadAccess, UserOperationalRegionAccess, MainheadCapability, TeamCapability, UserCapability, Capability, Team, TeamMember, User, Project, ProjectMembership, WorkPackage, WorkPackageAssignment, AssetType, Substation (master data — see decision points).

DELETE (18): DefectEvidenceImage, DefectTimelineEntry, Defect, InspectionItemResult, InspectionResult, InspectionImage, Image, SiteVisitAsset, OperationalSessionAsset, SiteVisitParticipant, SiteVisitUser, Inspection, Asset, SiteVisit, OperationalSession, InspectionTemplateItem, InspectionTemplateSection, InspectionTemplate.

1. Exact table dependency (deletion) order

Children to parents. "Why" cites the constraint that forces the position; CASCADE rows could be auto-removed by their parent but are deleted explicitly for auditable counts.

#	Table	Scope filter	Why this position
1	DefectEvidenceImage	defectId in tenant defects	child of Defect (CASCADE)
2	DefectTimelineEntry	defectId in tenant defects	child of Defect (CASCADE); includes COMMENT events
3	Defect	via tenant IIR	child of InspectionItemResult (CASCADE)
4	InspectionItemResult	inspectionId in tenant inspections	child of Inspection (CASCADE)
5	InspectionResult	inspectionId in tenant inspections	must precede InspectionTemplateItem (RESTRICT)
6	InspectionImage	inspectionId in tenant inspections	child of Inspection (CASCADE)
7	Image	tenantId = TENANT	independent; SET NULL refs — clear all media first
8	SiteVisitAsset	siteVisitId in tenant visits	must precede Asset (RESTRICT)
9	OperationalSessionAsset	sessionId in tenant sessions	must precede Asset (RESTRICT)
10	SiteVisitParticipant	siteVisitId in tenant visits	child of SiteVisit (CASCADE)
11	SiteVisitUser	siteVisitId in tenant visits	child of SiteVisit (CASCADE)

#	Table	Scope filter	Why this position
12	Inspection	tenantId = TENANT	must precede SiteVisit / Asset / Template (RESTRICT)
13	Asset	tenantId = TENANT	after RESTRICT refs (8, 9, 12) cleared
14	SiteVisit	tenantId = TENANT	after Inspection (RESTRICT) + cascade children
15	OperationalSession	workspaceId = TENANT	after OperationalSessionAsset + inspections
16	InspectionTemplateItem	templateId in tenant templates	after InspectionResult (RESTRICT) cleared
17	InspectionTemplateSection	templateId in tenant templates	child of InspectionTemplate (CASCADE)
18	InspectionTemplate	tenantId = TENANT	after Inspection (RESTRICT) + sections/items

2. Full production-safe SQL reset script

Tenant-scoped, single transaction, explicit ordering, one row count per statement. Run the deletes, inspect the printed counts and the Section 5 validation, then COMMIT (or ROLLBACK).

```
\set ON_ERROR_STOP on
\timing on
\set tenant_code 'ASCURE' -- <- set to the pilot Tenant.code

-- Resolve the tenant id into a psql variable; errors out if 0 or many rows.
SELECT id AS tenant_id FROM "Tenant" WHERE code = :tenant_code \gset
\echo '>> Resetting operational data for tenant_id = ' :tenant_id

BEGIN;

-- 1-2. Defect media + timeline (children of Defect)
DELETE FROM "DefectEvidenceImage" e
  USING "Defect" d, "InspectionItemResult" iir, "Inspection" i
  WHERE e."defectId" = d.id AND d."inspectionItemResultId" = iir.id
    AND iir."inspectionId" = i.id AND i."tenantId" = :tenant_id';
DELETE FROM "DefectTimelineEntry" t
  USING "Defect" d, "InspectionItemResult" iir, "Inspection" i
  WHERE t."defectId" = d.id AND d."inspectionItemResultId" = iir.id
    AND iir."inspectionId" = i.id AND i."tenantId" = :tenant_id';

-- 3. Defects
DELETE FROM "Defect" d
  USING "InspectionItemResult" iir, "Inspection" i
  WHERE d."inspectionItemResultId" = iir.id AND iir."inspectionId" = i.id
    AND i."tenantId" = :tenant_id';

-- 4-6. Inspection result rows + images (children of Inspection)
DELETE FROM "InspectionItemResult" iir
  USING "Inspection" i WHERE iir."inspectionId" = i.id AND i."tenantId" = :tenant_id';
DELETE FROM "InspectionResult" ir -- before InspectionTemplateItem (RESTRICT)
  USING "Inspection" i WHERE ir."inspectionId" = i.id AND i."tenantId" = :tenant_id';
DELETE FROM "InspectionImage" ii
  USING "Inspection" i WHERE ii."inspectionId" = i.id AND i."tenantId" = :tenant_id';

-- 7. All images for the tenant
DELETE FROM "Image" WHERE "tenantId" = :tenant_id';

-- 8-9. Asset link tables (RESTRICT -> Asset)
DELETE FROM "SiteVisitAsset" sva
  USING "SiteVisit" sv WHERE sva."siteVisitId" = sv.id AND sv."tenantId" = :tenant_id';
DELETE FROM "OperationalSessionAsset" osa
  USING "OperationalSession" os
  WHERE osa."operationalSessionId" = os.id AND os."workspaceId" = :tenant_id';

-- 10-11. Site visit membership (children of SiteVisit)
DELETE FROM "SiteVisitParticipant" p
  USING "SiteVisit" sv WHERE p."siteVisitId" = sv.id AND sv."tenantId" = :tenant_id';
DELETE FROM "SiteVisitUser" u
  USING "SiteVisit" sv WHERE u."siteVisitId" = sv.id AND sv."tenantId" = :tenant_id';

-- 12. Inspections (RESTRICT -> SiteVisit/Asset/Template; children already gone)
DELETE FROM "Inspection" WHERE "tenantId" = :tenant_id';

-- 13. Assets (all RESTRICT refs cleared above)
DELETE FROM "Asset" WHERE "tenantId" = :tenant_id';

-- 14. Site visits (Inspection RESTRICT + cascade children cleared)
DELETE FROM "SiteVisit" WHERE "tenantId" = :tenant_id';

-- 15. Operational sessions (children cleared)
DELETE FROM "OperationalSession" WHERE "workspaceId" = :tenant_id';

-- 16-18. Templates: items -> sections -> templates
DELETE FROM "InspectionTemplateItem" it
  USING "InspectionTemplate" t WHERE it."templateId" = t.id AND t."tenantId" = :tenant_id';
DELETE FROM "InspectionTemplateSection" s
  USING "InspectionTemplate" t WHERE s."templateId" = t.id AND t."tenantId" = :tenant_id';
DELETE FROM "InspectionTemplate" WHERE "tenantId" = :tenant_id';
```

```
-- >>> STOP. Run the Section 5 validation queries here (still inside the transaction).
-- >>> If everything is correct: COMMIT;
-- >>> If anything looks wrong: ROLLBACK;
```

Optional fast path (single-tenant only)

```
-- Optional single-tenant-only fast path (blunter; no tenant scoping, no per-table counts).
-- Safe here only because no KEEP table references these tables (no surprise CASCADE).
TRUNCATE
  "Defect", "DefectEvidenceImage", "DefectTimelineEntry",
  "InspectionItemResult", "InspectionResult", "InspectionImage", "Image",
  "SiteVisitAsset", "OperationalSessionAsset", "SiteVisitParticipant", "SiteVisitUser",
  "Inspection", "Asset", "SiteVisit", "OperationalSession",
  "InspectionTemplateItem", "InspectionTemplateSection", "InspectionTemplate"
RESTART IDENTITY;
```

Prefer the ordered DELETE script for auditability and tenant safety; use TRUNCATE only when wiping the entire single-tenant database is intended.

3. Backup command before reset

Full logical backup (custom format, restorable selectively), taken from the host:

```
# Whole-DB safety net (recommended)
docker exec ascore-postgres \
  pg_dump -U ascore -d ascore -Fc -f /tmp/ascore_pre_reset_20260604.dump
docker cp ascore-postgres:/tmp/ascore_pre_reset_20260604.dump ./ascore_pre_reset_20260604.dump

# Optional: targeted plain-SQL data dump of just the DELETE set (faster selective restore)
docker exec ascore-postgres bash -lc "pg_dump -U ascore -d ascore --data-only \
  -t '\"DefectEvidenceImage\"' -t '\"DefectTimelineEntry\"' -t '\"Defect\"' \
  -t '\"InspectionItemResult\"' -t '\"InspectionResult\"' -t '\"InspectionImage\"' \
  -t '\"Image\"' -t '\"SiteVisitAsset\"' -t '\"OperationalSessionAsset\"' \
  -t '\"SiteVisitParticipant\"' -t '\"SiteVisitUser\"' -t '\"Inspection\"' \
  -t '\"Asset\"' -t '\"SiteVisit\"' -t '\"OperationalSession\"' \
  -t '\"InspectionTemplateItem\"' -t '\"InspectionTemplateSection\"' -t '\"InspectionTemplate\"' \
  -f /tmp/ascore_delete_set_20260604.sql"
docker cp ascore-postgres:/tmp/ascore_delete_set_20260604.sql ./ascore_delete_set_20260604.sql

# Verify the dump before running the reset
pg_restore -l ./ascore_pre_reset_20260604.dump | head
```

4. Rollback strategy

- **Primary (in-transaction).** The whole reset is one transaction. Run Section 5 before committing; if counts are wrong -> ROLLBACK; and nothing changed. This is the real safety net — prefer it.
- **Post-commit, targeted restore.** Restore the dump into a scratch DB and re-insert the deleted tables in reverse dependency order (see below).
- **Post-commit, full restore.** Only if the reset just happened and nothing else changed: restore the whole dump over the database per your DR procedure (reverts kept-table changes too).

```
# Post-commit, targeted restore into a scratch DB, then re-insert in reverse dependency order.
docker exec ascore-postgres psql -U ascore -c 'CREATE DATABASE ascore_restore;'
docker exec ascore-postgres pg_restore -U ascore -d ascore_restore ./ascore_pre_reset_20260604.dump
# Re-insert parents first:
# InspectionTemplate -> Section -> Item -> OperationalSession -> SiteVisit -> Asset ->
# Inspection -> (SiteVisitUser/Participant/Asset, OperationalSessionAsset,
# InspectionImage/Result/ItemResult) -> Defect -> timeline/evidence -> Image
```

5. Validation queries after reset

5a. Delete set must all be 0 (tenant-scoped)

```
SELECT
  (SELECT count(*) FROM "SiteVisit" WHERE "tenantId"=:tenant_id) AS site_visits,
  (SELECT count(*) FROM "Inspection" WHERE "tenantId"=:tenant_id) AS inspections,
  (SELECT count(*) FROM "Asset" WHERE "tenantId"=:tenant_id) AS assets,
  (SELECT count(*) FROM "Image" WHERE "tenantId"=:tenant_id) AS images,
  (SELECT count(*) FROM "InspectionTemplate" WHERE "tenantId"=:tenant_id) AS templates,
  (SELECT count(*) FROM "OperationalSession" WHERE "workspaceId"=:tenant_id) AS sessions,
  (SELECT count(*) FROM "Defect" d
    JOIN "InspectionItemResult" iir ON iir.id=d."inspectionItemId"
    JOIN "Inspection" i ON i.id=iir."inspectionId"
    WHERE i."tenantId"=:tenant_id) AS defects,
  (SELECT count(*) FROM "SiteVisitAsset" sva
    JOIN "SiteVisit" sv ON sv.id=sva."siteVisitId"
    WHERE sv."tenantId"=:tenant_id) AS sv_assets,
  (SELECT count(*) FROM "OperationalSessionAsset" osa
    JOIN "OperationalSession" os ON os.id=osa."operationalSessionId"
    WHERE os."workspaceId"=:tenant_id) AS session_assets;
-- All columns must be 0.
```

5b. Keep set must be unchanged (snapshot before, compare after)

```
SELECT 'Organization' t, count(*) n FROM "Organization"
UNION ALL SELECT 'Branch', count(*) FROM "Branch"
UNION ALL SELECT 'OperationalRegion', count(*) FROM "OperationalRegion" WHERE "tenantId"=:tenant_id
UNION ALL SELECT 'Mainhead', count(*) FROM "Mainhead"
UNION ALL SELECT 'Project', count(*) FROM "Project"
UNION ALL SELECT 'WorkPackage', count(*) FROM "WorkPackage"
UNION ALL SELECT 'Team', count(*) FROM "Team" WHERE "tenantId"=:tenant_id
UNION ALL SELECT 'TeamMember', count(*) FROM "TeamMember"
UNION ALL SELECT 'User', count(*) FROM "User" WHERE "tenantId"=:tenant_id
UNION ALL SELECT 'UserMainheadAccess', count(*) FROM "UserMainheadAccess"
UNION ALL SELECT 'Capability', count(*) FROM "Capability"
UNION ALL SELECT 'AssetType', count(*) FROM "AssetType" WHERE "tenantId"=:tenant_id
UNION ALL SELECT 'Substation', count(*) FROM "Substation" WHERE "tenantId"=:tenant_id
ORDER BY t;
-- Compare to the pre-reset run of the same query: every row identical.
```

5c. Integrity sanity (should be 0 by construction)

```
SELECT
  (SELECT count(*) FROM "Inspection") AS inspections_total,
  (SELECT count(*) FROM "Project" p
    LEFT JOIN "Mainhead" m ON m.id=p."mainheadId"
    WHERE p."mainheadId" IS NOT NULL AND m.id IS NULL) AS projects_bad_mainhead,
  (SELECT count(*) FROM "Substation" s
    WHERE NOT EXISTS (SELECT 1 FROM "Asset" a WHERE a."substationId"=s.id))
    AS substations_without_assets;
-- inspections_total = 0; projects_bad_mainhead = 0;
-- substations_without_assets is informational (see decision points).
```

6. What data must be recreated after reset

Item	Why	How
Inspection Templates + sections + items (incl. checklist templates)	Fully deleted	Rebuild via Checklist Builder, or run the seed to recreate the baseline SAVR template + capability mappings (idempotent upserts — review seed first). Re-publish (ACTIVE/isActive) and re-map to AssetTypes/scope.
Asset master inventory	All Asset rows deleted	Re-import the asset register if pre-existing assets are needed; otherwise assets are created during visits.
Operational Sessions	All deleted	Recreated through the normal field workflow.
Pencawang / Substations	Kept by default	None needed unless you also prune them (decision points).
Dashboards / Defect board / QA queues	Derived from deleted data	Repopulate automatically as the new pilot generates visits, inspections, defects.
Projects / Work Packages	Kept	Verify mainheadId/links are the intended ones for the new pilot; legacy mainhead text columns persist.

Decision points to confirm before running

- **Pencawang / Substations** — not in the delete list, kept by default. The "New Pencawang" check-in flow can create them, so some may be pilot-created. To remove them too, prune after the reset using `5c.substations_without_assets` as the candidate set.
- **Projects / Work Packages** — spec says "if safe." They ARE safe to keep (no FK dependency on deleted data). Default = keep; confirm if you would rather wipe them too.
- **Templates via seed** — running the seed to restore the baseline template also upserts `org/branch/mainhead/admin/capability` rows. Confirm whether to use seed or rebuild templates manually.